gemstone-rs User Manual

Core Rust API usage for sessions, OOPs, browser operations, and transactions

Generated from the Markdown sources in gemstone-rs/docs.

Contents

User Manual

`gemstone-rs` is the safe Rust client API for GemStone/S over GCI. Rust code imports it as `gemstone_rs`.

Configuration

Use `Config::from_env()` for normal applications:

```
let config = gemstone_rs::Config::from_env()?;
```

Use the builder when you want explicit configuration:

```
use gemstone_rs::Config;

let config = Config::builder()
    .stone("gs64stone")
    .host("localhost")
    .netldi("netldi")
    .username("DataCurator")
    .password("swordfish")
    .build()?;
```

Sessions

`Session` logs out automatically on drop. Calling `logout()` explicitly is still useful in examples and command-line tools.

```
use gemstone_rs::{Config, Session, Value};

let mut session = Session::login(Config::from_env()?)?;
let value = session.eval("3 + 4")?;
assert_eq!(value, Value::SmallInt(7));
session.logout()?;
```

`Session` is deliberately not `Send` or `Sync`. Keep a session on the thread that logged it in.

Eval and Perform

`eval` returns a marshalled `Value`:

```
let value = session.eval("3 + 4")?;
```

`execute` returns a raw `Oop`:

```
let object_class = session.execute("Object")?;
```

`perform` returns a marshalled `Value`:

```
let seven = session.smallint_oop(7);
let printed = session.perform(seven, "printString", &[])?;
```

`perform_oop` returns a raw `Oop`:

```
let printed_oop = session.perform_oop(seven, "printString", &[])?;
let printed = session.fetch_string(printed_oop)?;
```

OOP and Value Conversion

`Value` covers nil, booleans, small integers, characters, fetched strings, and raw OOPs:

```
use gemstone_rs::Value;

let oop = session.value_to_oop(&Value::SmallInt(7))?;
let value = session.eval("true")?;
println!("{value:?}");
```

Helpers:

```
let nil = session.nil_oop();
let yes = session.bool_oop(true);
let seven = session.smallint_oop(7);
let text = session.new_string("hello")?;
let symbol = session.new_symbol("ExampleSymbol")?;
```

Globals

`global_get` and `global_put` use `UserGlobals`:

```
let text = session.new_string("hello from Rust")?;
session.global_put("GemStoneRsText", text)?;
```

```
let stored = session.global_get("GemStoneRsText")?;
println!("{}", session.fetch_string(stored)?);
```

Transactions

Use `transaction` when you want commit-on-success and abort-on-error:

```
session.transaction(|session| {
    let value = session.new_string("committed")?;
    session.global_put("GemStoneRsCommitted", value)
})?;
```

Manual transaction primitives are also available:

```
let needs_commit = session.needs_commit()?;
let in_transaction = session.in_transaction()?;
session.commit()?;
session.abort()?;
```

Export-Set Lifetime

Use `retain_oop` when a raw OOP must be held across calls and protected from GemStone-side collection:

```
let text = session.new_string("retained")?;
let handle = session.retain_oop(text)?;
println!("{}", handle.oop().raw());
handle.release()?;
```

The handle releases on drop if you do not call `release()`.

Browser API

```
use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);
let dictionaries = browser.dictionaries()?;
let classes = browser.classes("UserGlobals")?;
let protocols = browser.protocols("Object", false, "")?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "")?;
let source = browser.source("Object", "printString", false, "")?;
```

Pass an empty dictionary to resolve through the active user's symbol list.

Error Handling

Most APIs return `gemstone_rs::Result<T>`. Errors distinguish missing environment, missing config, GCI loading/calls, GemStone errors, illegal OOPs, unexpected typed codegen returns, and conversion issues.

```
match Config::from_env() {
    Ok(config) => println!("stone {}", config.stone),
    Err(err) => eprintln!("configuration error: {err}"),
}
```

This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.